IR tabanlı görsel üretim pipeline'ı

Mimari analiz ve öneriler

3 Kasım 2026 · Hazırlayan: Claude (Claude Code) · Durum: tartışma taslağı, 2. sürüm (inceleme düzeltmeleriyle); uygulama tartışmadan sonra Python ile yapılacak

Bu belge ne?

Hedeflenen mimarinin ayrıntılı incelemesi: konu promptu → planner → IR ajanı → IR → compiler → Manim kodu ve resim. Amaç, sistemi kontrol edilebilir, jenerik ve belirli konulara ya da tek bir kütüphaneye overfit olmamış hale getirmek için neyin değişmesi gerektiğini ortaya koymak ve karar verilmesi gereken noktaları listelemek.

Hızlı okuma: Özet (s. 2), Şekil 1 (s. 3) ve karar tablosu (§7, s. 16).

İncelenen kaynaklar

- `~/Downloads/ir3/docs` : IR.md, SEMANTICS.md, STATUS.md, ARCHITECTURE.md, FINDINGS.md ve örnekler (28 Ekim; referans alınan sürüm)
- `~/Documents/animations-pipeline-trys/building-IR/ir-to-picture/docs` : aynı belgelerin 18 Ekim tarihli eski alt kümesi
- `building-IR/plan.md` (ilk animasyon planı) ve `plans/*.txt` (200 doğal dil resim tarifi)
- ir3'ün Python hattı (`py/irpic`) ve eval özetleri (spec gap listeleri)

İçindekiler

Özet: en önemli 8 öneri 2

1 Pipeline ve sorumluluk sınırları 3

2 IR değişiklikleri 6

2.1 Canvas, birimler, en-boy oranları 6

2.2 Altyazı 7

2.3 Sayısal konumlar 7

2.4 Renkler 8

2.5 Tipografi 8

2.6 Bölge cebiri: subtraction, kesişimi boyama, maskeleme 9

2.7 Overfit'e karşı yapı 10

2.8 Gruplar, transform, çizim sırası, stiller 11

3 Compiler: "sadece compiler değişsin" 11

4 IR ajanı ve kod çalıştırma 13

5 3D hazırlığı ve çakışma semantiği 14

6 Korunması gerekenler 15

7 Karar vermeniz gereken noktalar 16

8 Uygulama sırası 16

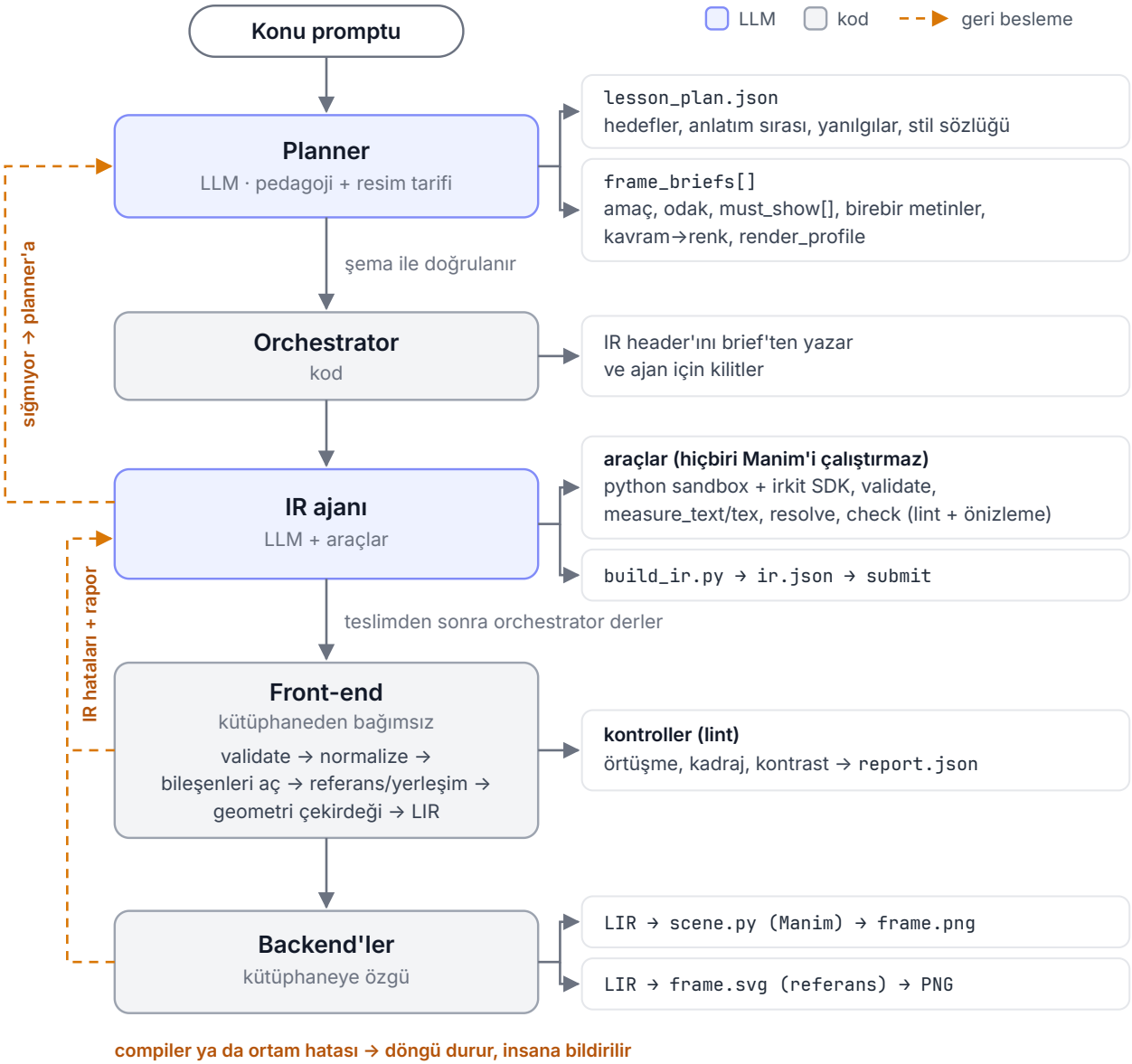
Ek: Terimler 17

Özet: en önemli 8 öneri

Aşağıdaki sekiz madde belgenin özeti; her birinin ayrıntısı işaret edilen bölümde.

1. **Tasarım felsefesi değişiyor; bunu bilerek yapın.** ir3 şu ilkeye dayanıyor: küçük model hata yapmasın diye yanlış değer hiç yazılamasın. Bu yüzden 10 renk token'ı, zone isimleri ve boyut token'ları var. Siz kod çalıştırma ve sürekli değerlere geçiyorsunuz; bu doğru bir yön. Ama token'ların baştan engellediği hatalar (okunmaz renk, minik yazı) artık yakalanmak zorunda: kontrast ve minimum boyut lint'leri, ölçüm araçları ve tema varsayılanları gerekiyor. §2.3–2.5, §2.8
2. **Canvas IR'nin içine girmeli; sizin tasarımınız da bunu öngörüyor.** Bu, ir3'teki "canvas IR'nin parçası değil, çalıştırma ayarı" kararının tersi; ir3'ten taşınacak kod buna göre değişmeli. Konumlar sayıyla verildiğinde her sayı canvas'a bağlı hale gelir; IR kendi canvas'ını taşımazsa başka bir canvas'ta bozulur. §2.1
3. **Planner'ın kararları IR'de standart ve kilitli bir yere girsin:** `{{FONT}}` gibi şablonlarla değil, orchestrator'ın brief'ten yazdığı bir header olarak. Nesneler değerleri kopyalamak yerine header'daki adlara referans versin: punto rolü (`"size": "label"`), kavram rengi (`"color": "a"`), stil sınıfı (`"style": "muted"`). §1, §2.1
4. **Compiler iki parçaya ayrılsın.** Kütüphaneden bağımsız bir front-end doğrulama, ölçüm, yerleşim, geometri ve kontrolleri yapar; çıktısı sürümlü bir LIR'dir (çözülmüş çizim listesi). Bunu basit çiziciler (backend'ler) okur: Manim kod üretici ve bir SVG referansı. Yeni bir kütüphane eklemek, yalnızca yeni bir backend ve gerekirse onun metin ölçüm sağlayıcısını yazmak demek; "sadece compiler değişsin" hedefini gerçekten sağlayan şey bu sınır. §3
5. **Bölge cebiri eklensin:** subtraction, intersection, union, clip ve boya kovası gibi çalışan bir dolgu (`flood_fill`). Bunlar backend'de değil, front-end'in geometri çekirdeğinde (shapely) hesaplansın; backend'e hazır, gerekirse delikli path olarak gitsin. §2.6
6. **Overfit'e karşı küçük bir çekirdek kurun;** üstüne bir bileşen kütüphanesi ve `defs` /asset desteği gelsin. Konuya özel yeni tipler (yay/spring, pil, pusula) eklemek yerine ajan bu geometriyi kodla üretsin. §2.7
7. **IR ajanı JSON'u elle yazmasın;** Python builder SDK kullanan bir script yazsın (`build_ir.py` → `ir.json`). Böylece sayılar elle kopyalanmaz, hesaplanır. Sözleşme yine JSON olarak kalır. §4
8. **3D için şimdiden kancalar bırakın:** bir `space` soyutlaması, LIR'de derinlik bilgisi ve çakışmayı tek bir "ekranda örtüşme" kontrolü yerine üç soruya bölen bir kontrol modeli: metin okunuyor mu (ekranda), şekiller gerçekten iç içe mi (3D'de dünya uzayında), temas kasıtlı mı (IR'deki referanslardan). §5

1 Pipeline ve sorumluluk sınırları



Şekil 1. Önerilen akış. Lila kutular LLM, gri kutular kod. Ajan IR'yi teslim eder; derlemeyi ve render'ı orchestrator çalıştırır. Kesikli turuncu oklar geri besleme: IR kaynaklı hatalar ajana IR terimleriyle döner; compiler ya da ortam hataları ajana değil insana gider (§1); brief kısıtlara sığmıyorsa ajan planner'a döner.

Yeni bir bileşen: orchestrator

Akışınızda adımları birbirine bağlayan ve hata döngüsünü yöneten parça tanımlı değil. Öneri, bunu açık bir bileşen yapmak: orchestrator, LLM değil deterministik koddur. Görevleri: brief'i şemayla doğrulamak; header'ı (canvas, güvenli alan, altyazı bandı, punto ölçeği) brief'ten hesaplayıp kilitlemek; açılışta font ve LaTeX kurulumunu kontrol etmek; backend'i proje ayarlarına göre seçmek; ajanı çalıştırıp tur sınırını uygulamak; ajan IR'yi teslim edince front-end'i, backend'i ve render'ı çalıştırmak; hataları aşağıdaki tabloya göre ajana, planner'a ya da insana yönlendirmek; çıktı klasörünü yazmak.

Planner çıktısı yapılandırılmış olmalı

`plans/*.txt` dosyaları serbest bir paragraf ve bir renk listesinden oluşuyor; ir3'ün örnek prompt'larındaki (`docs/examples`, `prompts/`) "Must show" listelerinde de ID yok. Bu haliyle bir program kontrol edemez. Önerilen `frame_brief` şeması:

alan	içerik
`purpose`	resmin pedagojik rolü: tanıtma, tanım, örnek, karşı örnek, özet
`intent`, `focus`	izleyicinin ne anlaması gerektiği; gözün ilk nereye gideceği
`must_show`	`[{id, text, data?}]`: resimde mutlaka olması gerekenler, kontrol edilebilir ID'lerle
`texts`	başlık, etiket ve altyazı metinleri; hedef dilde, birebir
`concept_colors`	kavram → renk
`constraints`	kısıtlar, `must` / `should` ayrımıyla
`render_profile`	canvas (oran, çözünürlük, platform, kenar payları), altyazı, tipografi, tema; ayrıntısı §2.1
`composition_notes`	doğal dildeki resim tarifi

Ders düzeyinde bir stil sözlüğü

Kavram → renk eşlemesi, notasyon, tipografi ve canvas genelde ders boyunca değişmez. Planner bunları `lesson_plan.json`'da bir kez belirlesin, her frame miras alsın; frame düzeyinde override serbest olsun. Çelişki durumunda öncelik sırası: **kullanıcının açık isteği > proje ayarı > planner'ın seçimi.** Proje ayarı, ekibin pipeline için sabitlediği varsayılanlar demek (ör. hedef platform, font beyaz listesi).

Planner'ın kararları: şablon yerine kilitli header

JSON içine konan `{{FONT}}` gibi şablonların üç sorunu var: yerine konmadan doğrulanamazlar, ajan onları bozabilir ve her nesnede tekrar ederler. Önerilen düzen:

- Orchestrator header'ı brief'ten yazar: brief'teki `intent` cümlesi → `intent`; `render_profile`'ın canvas ve altyazı kısmı → `frame`, tipografisi → `typography`, teması → `frame.background` ve `styles`; `concept_colors` → `colors`.
- Ajan yalnızca gövdeyi yazar: `focus` (odak nesnelerinin id'leri), `extra_colors`, `zones`, `spaces`, `defs`, `objects`, `allow`.
- Validator, teslim edilen `ir.json`'daki header'ın orchestrator'ın yazdığı header ile birebir aynı olduğunu hash ile kontrol eder.
- Nesneler header'daki adlara referans verir (`"size": "label"`, `"color": "a"`, `"style": "muted"`). Gerektiğinde sayısal değer de yazabilirler; o zaman minimum punto ve "beyan edilmemiş renk" lint'leri devreye girer (§2.4, §2.5).

Planner'a geri dönüş yolu

Ajan kilitli kısıtlarla brief'i karşılayamıyorsa (örneğin 9:16 + altyazı + çok metin), içeriği sıkıştırmak yerine orchestrator üzerinden planner'a yapılandırılmış bir istek gönderebilmeli: "metni kısalt" ya da "iki resme böl". Kontrol edilebilirliğin önemli bir parçası bu.

Döngü ve hata yönlendirme

Ajan IR'yi teslim edince (`submit`) derlemeyi orchestrator çalıştırır: önce front-end, sonra backend ve render. Döngü yalnızca derleme hatasında dönmesin. Bir resim ancak doğrulamadan geçtiğinde, derlendiğinde, hiç hata kalmadığında, her uyarı düzeltildiğinde ya da `allow` ile gerekçesiyle kabul edildiğinde ve (ileride)

görsel eleştirmen de onayladığında hazır (READY) sayılsın; bu, ir3'teki READY tanımının genişletilmiş hali. Görsel eleştirmen, son resme bakıp okunabilirliği ve pedagojik amaca uygunluğu değerlendirecek bir görsel dil modeli. Hatalar kaynağına göre farklı yerlere gitmeli:

kaynak	örnek	nereye gider
`ir`	şema, referans, boş kesişim, LaTeX hatası, örtüşme	IR ajanına: kod + JSON yolu + nesne id + ipucu
`capability`	seçilen backend bu özelliği desteklemiyor	önce otomatik olarak basit primitive'lere indirilir; olmazsa ajana
`internal` / `environment`	compiler bug'ı, eksik font, LaTeX kurulu değil	döngü durur, insana bildirilir; ajan bunu düzeltemez, boşuna döner
`infeasible`	brief kısıtlara sığmıyor	planner'a

`ir` hatalarının çoğu, ajan daha teslim etmeden kendi araçlarıyla (§4) yakalanır; teslimden sonra dönenler istisna olmalı. Ajan hiçbir zaman Manim traceback'i görmemeli; her hata IR terimleriyle dönmeli. ir3'teki iki mekanizma olduğu gibi kalsın: tek bir issue-code kaydı (her hata ve uyarı türünün tek yerde tanımlı kodu, ör. `E-SUBTITLE`) ve `allow` (ajan bir uyarıyı gerekçesini yazarak bilinçli olarak kabul eder, ör. kasıtlı bir örtüşme). ir3'teki tur sınırı (varsayılan 3) ve "ilerleme yoksa dur" kuralı da kalsın. Yeni olarak her turun IR'si saklansın ve en az açık sorunu olan sürüm teslim edilsin; ir3'te son tur ne bıraktıysa o kalıyor.

Çıktı klasörü

dosya	içerik
`frame.png`	üretilen resim (Manim)
`frame.svg`	SVG referans backend'inin çıktısı; karşılaştırma ve vektör kullanım için
`ir.json`	ajanın yazdığı IR
`scene.py`	üretilen Manim kodu
`brief.json`	planner'ın bu resim için verdiği brief
`build_ir.py`	ajanın IR'yi üreten script'i
`lir.json`	çözülmüş çizim listesi
`report.json`	doğrulama ve lint sonuçları
`meta.json`	IR, compiler, Manim ve font sürümleri ile metin ölçüm sağlayıcısı; aynı resmi tekrar üretebilmek için
`alt.txt`	erişilebilirlik metni; IR'deki `intent` ve metinlerle brief'teki `must_show` listesinden otomatik üretilir

2 IR değişiklikleri

2.1 Canvas, birimler, en-boy oranları

ir3'ün birim kuralı korunmaya değer: kısa kenar 9 birim, 1080 px'te 1 birim = 120 px, orijin merkezde, y yukarı doğru. Bu kural her orana temizce genelleşir ve eski run'larla uyumlu kalır: 16:9 → 16×9, 9:16 → 9×16, 4:3 → 12×9, 1:1 → 9×9. Canvas herhangi bir W:H oranı olabilsin; aşağıdaki hazır liste doğrulama ve arayüz için dursun.

oran	tipik kullanım	piksel (kısa kenar 1080)	birim
16:9 · 9:16	YouTube, sunum · Shorts, Reels, TikTok	1920×1080 · 1080×1920	16×9 · 9×16
4:3 · 3:4	klasik slayt, tablet	1440×1080 · 1080×1440	12×9 · 9×12
1:1	kare gönderi	1080×1080	9×9
4:5	Instagram dikey gönderi	1080×1350	9×11.25
3:2 · 2:3	fotoğraf, Pinterest	1620×1080 · 1080×1620	13.5×9 · 9×13.5
16:10	laptop ekranı	1728×1080	14.4×9
21:9	ultra geniş	2520×1080	21×9
1.91:1	link önizleme görseli	2064×1080	17.2×9
1:√2	A4 çalışma kağıdı (baskı)	1080×1528	9×12.73

- **Piksel boyutları çift sayıya yuvarlansın:** H.264 gibi yaygın video kodlayıcıları, kullandıkları renk örneklemesi (yuv420p) yüzünden tek sayılı genişlik ya da yükseklik kabul etmez.
- **En-boy oranı ile platformun güvenli alanı ayrı şeyler.** ir3'ün 9:16 güvenli alanı asimetrik: x −4…3.5, üstte ve altta geniş pay. Bu, Reels/TikTok arayüzü yüzünden böyle; oranın değil platformun özelliği. Öneri: `render_profile = {aspect, resolution, platform, margins, subtitle, typography: {font_pair, scale, lang}, theme}`. Güvenli alan = canvas − kenar payları − platform overlay'leri − altyazı bandı. Bunları orchestrator hesaplayıp header'a yazsın.
- **Zone'lar,** konumlar sayıyla verilince ana konumlandırma aracı olmaktan çıkıyor. SDK'da yardımcı olarak kalabilirler; IR'de ise isteğe bağlı, sayısal `zones: {"plot": [x0, y0, x1, y1]}` olarak tanımlansınlar. Adları `zones` kalsın; §2.6'daki `region` başka bir kavram.
- **Overfit uyarısı:** mevcut 200 planın 198'i dikey. Yeni oranları test etmek için eval korpusunun da çeşitlenmesi gerekiyor.

Header'ın nasıl görüneceğine dair bir taslak:

```
{
  "version": "frame-ir/1.0",
  // header (intent, frame, typography, colors, styles): orchestrator brief'ten yazar; ajan
değiştiremez
  "intent": "Bir vektörün başka bir vektör üzerine izdüşümü",
  "frame": {
    "aspect": "4:5", "pixels": [1080, 1350], "units": [9, 11.25],
    "platform": "instagram_feed", "background": "#0E1117",
    "subtitle": {"mode": "reserve", "lines": 2, "band": [-4.2, -5.3, 4.2, -4.1]},
    "safe": [-4.1, -3.8, 4.1, 5.2]
  },
  "typography": {
    "lang": "tr", "text_font": "Inter", "math": "latin-modern",
    "sizes": {"title": 64, "heading": 52, "body": 42, "label": 38, "caption": 34}  // px, kısa kenar
1080'e göre
  },
  "colors": {"a": "#FFD43B", "b": [88, 196, 221], "proj": "#83C167"},
  "styles": {
    "muted": {"opacity": 0.35},
    "construction": {"dash": [14, 10], "stroke_width": 3}
  },
  // gövde: ajan yazar
  "focus": ["proj_ab"], "extra_colors": {}, "zones": {},
  "spaces": [], "defs": [], "objects": [], "allow": []
}
```

2.2 Altyazı

Altyazı için üç mod:

mod	anlamı
`none`	bant yok; güvenli alanın tamamı içeriğe kalır
`reserve`	bant boş bırakılır; altyazı sonradan, video kurgu aşamasında (ör. seslendirmeyle senkron) eklenir
`burn_in`	front-end, brief'teki altyazı metnini `caption` rolüyle banda çizer

- **Bant yüksekliği sabit olmasın:** satır sayısı × satır aralığı oranı (ör. 1,2) × caption puntosu + dolgu ile hesaplansın. Sonuç referans px'tir; birime çevirmek için 120'ye bölünür.
- **Konumunu platform belirlesin.** Dikey sosyal medyada en altta platform arayüzü olduğu için bant onun üstüne konur; arayüzü resmin üstüne binmeyen platformlarda (YouTube, sunum) en alta.
- ir3'teki `E-SUBTITLE` kontrolü (bantta bir şey çizilirse hata) bu hesaplanmış banda göre çalışsın; `burn_in` modunda front-end'in banda çizdiği altyazı bu kontrolün dışında kalır.
- Altyazı metnini planner yazsın.

2.3 Sayısal konumlar

Geometri ve kompozisyonun sayılarla verilip kodla hesaplanmasına katılıyorum. Yine de ir3'teki şu üç şey korunmalı:

1. **Koordinat uzayları (`in`).** "Sayıyla konum" illa ekran koordinatı demek değil. Matematik nesneleri kendi koordinat sisteminde tarif edilsin: `"in": "P"` alanı olan bir vektörün `[3, 1]` koordinatı, `P` düzleminin birimleriyle okunur. Ekrana dönüştürmek uzayın işi olsun. 3D'nin temeli de bu.
2. **Anchor referansları** (`"v.tip"`, `{"on": "f", "x": 2}`). Bunlar kelimeyle konum vermek değil, hesaplanmış noktalara referans. Düzeltme turunda bir nesne oynatıldığında ona bağlı her şey birlikte hareket eder. İleride animasyonda da gerekli: bir nesne hareket ederken ona referansla bağlı her şey (etiket, ok ucu) onu kendiliğinden izler; sayıyla kopyalanmış konumlar izlemez.

3. **Metnin ölçüme dayalı yerleşimi.** Metnin genişliği fonta ve onu çizen motora bağlı; mutlak sayıyla yerleştirilen metin, ölçümün yapıldığı motora bağımlı kalır ve başka bir backend'de kayar. Çözüm üç parçalı:
   - SDK'da `measure_text` ve `measure_tex` araçları olsun (§3, "Metin ölçümü"); ajan tahmin etmesin, ölçsün.
   - Metin kutunun ortasından değil bir anchor noktasından konumlansın (`anchor: "left_baseline"`); böylece genişlik farkı sadece uzak uca yansır.
   - `next_to` gibi ilişkiler opsiyonel olarak kalsın.

Kelimeyle verilen diğer seçenekler sürekli değer alabilsin, isimler kısayol olarak kalsın. Örneğin `label_dir` için `"up_left"` , `135` (derece) veya `[dx, dy]` yazılabilsin; `label_offset` ve boyutlar sayı ya da header'daki bir ad (ör. `"label"`) olabilsin. Birimler ir3'teki gibi kalsın: konum ve uzunluk sahne biriminde, punto ve çizgi kalınlığı referans px'te (kısa kenar 1080 iken).

2.4 Renkler

- **Format:** `"#RRGGBB"` veya `[r, g, b]` (0–255, sRGB). Opaklık rengin içine (alfa kanalı) değil, ayrı bir `opacity` alanına yazılsın; aynı şeyi iki yoldan söylemek mümkün olmasın. Saydam renklerin nasıl birleştirileceği SEMANTICS.md'de sabitlensin.
- `colors` **eşlemesi (kavram → renk) korunsun,** sadece değerleri RGB olsun. Nesnede doğrudan renk yazmak serbest kalsın, ama "beyan edilmemiş renk" uyarısı versin. "Bir kavram, bir renk" ilkesi böylece korunur.
- **Ajan ek renk tanımlayabilsin.** Kilitli `colors` planner'ın kavram renkleri. Kesişim gibi türetilmiş bölgeler için ajan gövdede ek renkler tanımlayabilsin (`extra_colors`), ör. `{"AB": {"mix": ["a", "b"]}}` ya da doğrudan RGB. Bunlar "beyan edilmemiş renk" sayılmaz ama kontrast ve ΔE lint'lerinden geçer.
- **Token'ların verdiği garantinin yerine lint:**
  - metin ile arka plan arasındaki kontrast (WCAG oranı, metnin altındaki gerçek renge göre),
  - kavram renklerinin birbirinden ayırt edilebilirliği (ΔE),
  - opsiyonel renk körlüğü simülasyonu.
- **Grup opaklığının anlamı açıkça tanımlanmalı.** Manim'de `set_opacity` her çocuğa ayrı uygulanır, bu yüzden üst üste binen çocuklar birbirinin içinden görünür. SVG ise grubu tek bir katman olarak birleştirir. Öneri: çocuk başına çarpma kuralı (front-end grubun opaklığını her çocuğun opaklığıyla çarpar; LIR'e grup opaklığı gitmez) ve çocukları örtüşen yarı saydam gruplarda uyarı. Backend'lerin ayrıştığı tipik bir nokta bu; SEMANTICS.md disiplini sürsün.

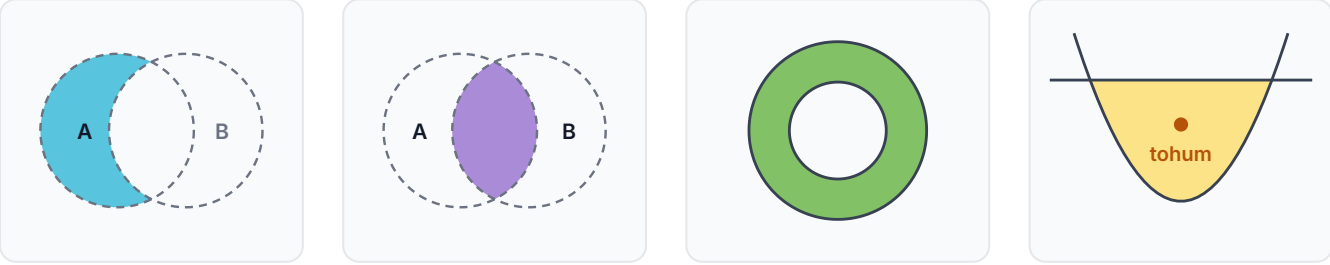
2.5 Tipografi

- **Planner tek bir punto değil, bir tip ölçeği seçsin:** taban punto + oran ya da `compact` , `comfortable` , `large` gibi bir hazır ayar. `title` , `heading` , `body` , `label` ve `caption` rollerinin puntolarını orchestrator hesaplasın. Planner tek punto verirse ajan geri kalanını uydurur.
- **Metin fontu ile matematik fontu ayrı kaynaklardan gelir.** Manim'de `Text` Pango ile sistem fontunu kullanır, `MathTex` ise LaTeX şablonunu. Planner, kurulu ve lisansı uygun "metin fontu + uyumlu matematik paketi" çiftlerinden oluşan bir beyaz listeden seçsin. Orchestrator açılışta bu fontların kurulu olduğunu kontrol etsin.
- **Header'da** `lang` **alanı olsun** (ör. `"tr"`). Satır kırma, font kapsama kontrolü ve LaTeX şablonu bunu kullanır. Türkçe karakterler için LaTeX şablonuna T1 font kodlaması (veya XeLaTeX + fontspec) eklenmeli.
- **Minimum punto,** kanvasın kısa kenarına göre lint'lensin.

2.6 Bölge cebiri: subtraction, kesişimi boyama, maskeleme

İki isteğiniz (şekil çıkarma/maskeleme ve kesişimleri boyamak için içini doldurma) aslında aynı eksik kavramın parçaları: **bölge (region)**. Şekil 2 dört örneği gösteriyor.

```
"defs": [
  {"id": "A", "type": "circle", "center": [-0.6, 0], "radius": 1.5},
  {"id": "B", "type": "circle", "center": [0.6, 0], "radius": 1.5},
  {"id": "outer", "type": "circle", "center": [0, -3], "radius": 1.2},
  {"id": "inner", "type": "circle", "center": [0, -3], "radius": 0.7}
],
"objects": [
  // f, g, wall ve grid başka yerde tanımlı nesneler. Renkler kısalık için doğrudan yazıldı;
  // gerçek IR'de header'daki adlar ya da extra_colors kullanılır.
  {"id": "A_only", "type": "region", "shape": {"difference": ["A", "B"]}, "fill": "#58C4DD"},
  {"id": "AB", "type": "region", "shape": {"intersection": ["A", "B"]}, "fill": "#A98BD8"},
  {"id": "ring", "type": "region", "shape": {"difference": ["outer", "inner"]},
   "fill": "#83C167"},
  {"id": "between_fg", "type": "flood_fill", "seed": [1.2, 0.4], "bounded_by": ["f", "g", "wall"],
   "fill": "#FFD43B", "opacity": 0.3},
  {"id": "grid_in_A", "type": "group", "children": ["grid"], "clip": "A"}
]
```



difference: A − B
A ve B yalnızca operand (defs)

intersection: A ∩ B
kesişim kendi rengiyle boyanır

halka: dış − iç
delikli path (even-odd)

flood_fill: tohum
sınırlar açık eğri olabilir

Şekil 2. Bölge cebirinin dört kullanımı. Hepsi front-end'de geometri olarak hesaplanır ve backend'e hazır path olarak gider; halkadaki gibi gerekirse delikli. Kesikli çizgiler yalnızca operand olan (defs) şekilleri gösterir; bunlar çizilmez.

yapı	ne yapar
`region`	`union`, `intersection`, `difference`, `xor` işlemlerini iç içe ifadelerle yazar. Operand: kapalı bir şekil, başka bir bölge, yarı düzlem, bir grafiğin altı ya da iki grafik arası. ir3'teki `area` tipi bunun özel bir hali olur.
`flood_fill` (boya kovası)	Verilen sınır nesnelerinin çizgileri düzlemi parçalara böler; tohum noktasının düştüğü parça boyanır. Sınırların kapalı şekil olması gerekmez (parabol + doğru + eksen). Tohum bir sınırın üstündeyse ya da parça sınırsızsa anlaşılır bir hata verilir (ör. yeni bir `E-REGION` kodu).
`clip` / `clip_out`	Bir nesne veya grup bir bölgenin yalnızca içinde (`clip`) ya da yalnızca dışında (`clip_out`) çizilir. `clip_out`, sizin "maskeleme" dediğiniz işin bir biçimi; SVG'deki yumuşak (alfa) maske değil. Çizgiler ve dolgular geometrik olarak kırpılır; metin ve raster görsel kırpılmaz, denenirse uyarı verilir.
`defs`	Operand olarak kullanılan şekiller burada tanımlanır; kendileri çizilmez.

Spec gap'lerde ve STATUS.md'nin açık konularında istenen halka, elips dilimi, Venn bölgeleri ve yarısı aydınlık gezegen bu tek mekanizmayla karşılanır.

- **Hesaplama yeri: front-end'in geometri çekirdeği.** Boolean işlemler, bölgeleri ayırma ve kırpma için shapely/GEOS yeterli; eğrileri korumak gerekirse skia-pathops eklenebilir. Sonuç LIR'e hazır path olarak gider (gerekirse delikli). Üç faydası var:
  - her backend aynı şeyi çizer;
  - kontroller gerçek bölgeyi görür;
  - Manim'in boolean sınıflarına ya da başka bir kütüphanenin clip desteğine bağımlı kalmazsınız.
- **Renk karışımı için kesişimi açıkça boyayın.** İki saydam rengin üst üste binmesiyle oluşan renk çizim sırasına ve arka plana bağlıdır, çoğu zaman bulanık ve düşük kontrastlı çıkar ve kavram renkleri arasında yer almaz. Kesişimi kendi rengiyle boyamak rengi tam olarak kontrol edilebilir yapar; kontrast lint'i de bu rengi görür. Çarpma (multiply) gibi blend mode'lar kütüphaneler arasında aynı çalışmadığı için onlara da güvenilmesin.

2.7 Overfit'e karşı yapı

ir3'te tipler konu ihtiyacına göre eklendi, sonra iki kapalı listeye bağlandı: seviye 1 temel şekiller, seviye 2 bileşenler (tangent, secant, area, bars, network, vector_field, contour...). Seviye 3 (figürler) hiç başlamadı; listeler kapandıktan sonra gelen istekler yalnızca not edildi. Spec gap'ler daha fazlasını istiyor: yay (spring), tarama çizgisi, mantık kapısı, pil, pusula, insan figürü. En büyük gap grupları eğriler, figür/ikon ve şekiller. Her ihtiyaç için kapalı listeye yeni tip eklemek overfit'e götürür; hiç eklememek de ajanı çaresiz bırakır. Önerilen üç katmanlı yapı ir3'ün seviyelerinden iki noktada ayrılıyor: bileşenler kapalı bir liste değil, eklenti paketleri olur; üçüncü katman figür tipleri değil, ajanın kodla ürettiği geometridir.

katman	içerik	kim bilir
1. Çekirdek (küçük, kapalı)	path (düz çizgi + Bézier, birden çok alt yol, fill rule), temel şekiller, text, math, group, stack, region, flood_fill, space (`spaces` bloğunda), label, marker (eğrinin ortasındaki ok ucu dahil), image / SVG asset, `use`	IR şeması ve front-end; backend'ler bunları değil, LIR'e indirilmiş hallerini görür
2. Bileşenler (açık, eklenti)	axes, graph, brace, angle mark, table, network, vector_field, contour, bars... Her biri kendi şeması olan ve çekirdek nesnelere açılan saf bir Python fonksiyonu. Yeni alan (kimya, devre, biyoloji) = yeni bileşen paketi; çekirdek değişmez.	yalnızca front-end; açıldıktan sonra çekirdek nesnelere dönüşür
3. Ajanın kendi geometrisi	Kod çalıştırma sayesinde yay zikzağı, alan çizgileri (sayısal integrasyon), log eksen konumları, döşemeler çekirdek path ve gruplar olarak üretilir. Tekrarlanan figürler `defs` + `use` ile çoğaltılır. Figürler için lisanslı bir SVG sembol paketi; SVG'ler front-end'de path'e çevrilir.	ajan + SDK; IR'ye çekirdek path ve gruplar olarak girer

- **Anlam kaybolmasın:** üretilen path'ler için her nesneye serbest bir `tags` alanı (ör. `"tags": ["spring"]`). Çizimi değiştirmez, ama kontroller, görsel eleştirmen ve ileride animasyon kullanır (ör. `spring` etiketli bir path için sıkışma hareketini seçmek).
- **Statik kapsam kontrolü:** `covers: ["ms3"]` alanı ile her `must_show` maddesinin en az bir nesneyle karşılandığı kontrol edilebilir.

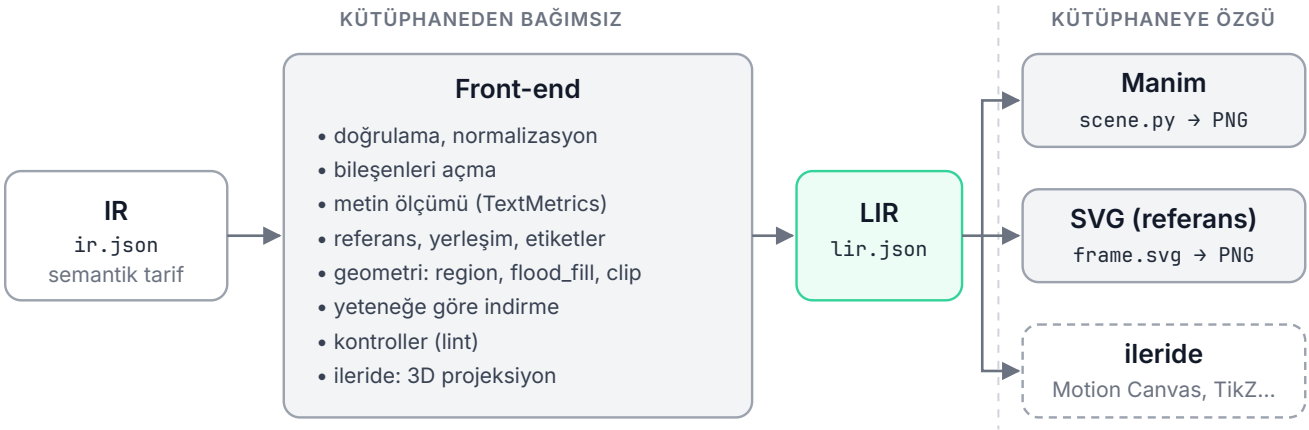
2.8 Gruplar, transform, çizim sırası, stiller

- **Grup kavramını ikiye ayırın.** ir3'teki `group` yalnızca kutu nesnelerini (text, tex, cells, table, iç içe group) satır ya da sütun halinde dizen bir yerleşim aracı; çember, dikdörtgen, path gibi geometriyi alamaz. Öneri:
  - `group` : her türden nesneyi alır; translate, rotate, scale ve matrix uygular; ortak opaklık ve clip taşır.
  - `stack` : satır, sütun veya ızgara yerleşimi yapar.

  "Bir figürün parçaları birlikte dursun" gap'i böylece kapanır.
- **Çizim sırası.** Dizi sırası çizim sırası olarak kalsın. Buna ek olarak opsiyonel sayısal bir `z` olsun; eşitlikte dizi sırası geçerli olsun. Gerekçe: bir bileşen dizide tek eleman olduğu halde birden çok parçaya açılır (ör. eksenlerin ızgarası, çizgileri ve sayıları); bu parçaların başka nesnelerin arasına girebilmesi için dizi sırası yetmez. 3D'de de derinliğe göre sıralamak gerekecek.
- **Stiller CSS gibi katmanlı olsun:** tema (arka plan, varsayılan metin ve çizgi rengi, varsayılan çizgi kalınlığı) → `styles` sınıfları (`muted` , `construction` , `emphasis`) → nesnenin kendi özellikleri. Öncelik sırası SEMANTICS.md'de yazılı olsun.

3 Compiler: "sadece compiler değişsin"

ir3'ün Python hattı doğru sınırı aslında zaten çizmiş: validate → resolve → resolved scene → drawer. ir3'te iki backend var: önce Motion Canvas yazılmış, sonra Manim; bugün ana backend Manim. STATUS.md'deki dersler, yerleşim backend'e bırakıldığında aynı IR'nin iki farklı resme dönüştüğünü gösteriyor (formül ölçeği, satır yüksekliği, baseline). Bu sınırı resmileştirmek gerekiyor; Şekil 3 önerilen katmanları gösteriyor.



Şekil 3. Katmanlar. Anlam ve yerleşim front-end'de hesaplanır; backend'ler yalnızca LIR'i çizer. Her backend desteklediği özellikleri bildirir, desteklemediklerini front-end daha basit primitive'lere indirir. Motion Canvas ir3'te zaten bir backend olarak var; yeni mimaride o da LIR'i çizen bir backend olur.

Front-end tek ve ortak olsun

Şunları yapar: doğrulama; normalizasyon (varsayılanlar, ifadeler, renkler); bileşenleri açma; metin ölçümü; referansları çözme, yerleşim ve etiket yerleşimi; geometri çekirdeği; ileride 3D projeksiyon; seçilen backend'in yeteneklerine göre indirme (lowering); kontroller.

LIR sürümlü ve belgelenmiş olsun

ir3'teki 5 primitive (`poly`, `dot`, `rrect`, `text`, `tex`) yetmez: delik, birden çok alt yol, fill rule, transform, Bézier, dash dizisi ve derinlik eksik. Semantik model olarak SVG'nin bir alt kümesini öneriyorum:

primitive	içerik
`path`	alt yollar (düz + Bézier); dolgu: renk, opaklık, fill rule; çizgi: renk, opaklık, kalınlık, dash, cap, join; opsiyonel şekil ipucu (`circle`, `rect`, `line` …)
`text`	metin, font, punto, renk, anchor, ölçülmüş kutu ve baseline
`math`	TeX kaynağı, punto, renk, ölçülmüş kutu; ayrıca front-end'in ölçüm sırasında derlediği glif path'leri
`image`	raster görsel; SVG asset'ler front-end'de path'e çevrilir
`group`	kaynak IR nesnesinin id'si, transform, `z`

- Front-end LIR'i son çizim sırasına göre dizer (`z` ve ileride 3D derinlik dahil); backend sırayı değiştirmez. Her primitive ayrıca opsiyonel bir `depth` alanı taşıyabilir; 3D'de örtülme kontrolleri ve animasyon bunu kullanır (§5).
- Grup opaklığı ve clip LIR'e gelmeden front-end'de uygulanır: opaklık çocuklara çarpılır (§2.4), clip geometrik olarak hesaplanır (§2.6). Böylece backend'ler arasındaki farklar sonuca yansımaz.
- Şekil ipucu, backend'in kütüphanenin kendi sınıfını (ör. Manim'de `Circle`) kullanabilmesi için var; çizilen geometri yine path'teki sayılarla aynı olmalı.
- `math`'teki glif path'leri sayesinde LaTeX'i olmayan bir backend de formülü çizebilir; Manim backend'i okunur kod için yine `MathTex` kullanır.

Her IR nesnesi LIR'de kendi id'siyle bir grup olsun

Front-end bir IR nesnesini çoğu zaman birden çok primitive'e açar: `axes` onlarca çizgi ve sayıya, bölge cebiri yeni path'lere, bir etiket yazı ve arka plan kutusuna dönüşür. Bu primitive'ler kaynak nesnenin id'sini taşıyan bir grupta toplanmazsa kimlik bu adımda kaybolur ve sonradan geri bulunamaz. Grup üç ayrı iş için gerekli; yalnızca biri hatalarla ilgili:

- **Geri bildirimi IR diliyle vermek.** Lint'ler LIR üzerinde çalışır. Grup yoksa rapor "`text#12`, `path#31` ile çakışıyor" der ve ajan bununla bir şey yapamaz; grup varsa "`label_vertex`, `parabola` ile 0.18 birim çakışıyor" der ve ajan doğrudan o nesneyi düzeltir. Hata olmayan geri bildirimler de aynı eşlemeyi kullanır: `resolve` tablosu, ölçümler ve önizlemedeki id etiketleri.
- **Okunur kod.** `scene.py`'deki değişken adları IR id'leriyle aynı olur (`parabola = …`, `label_vertex = MathTex(…)`); SVG'de `<g id="parabola">`. Dosyayı açan biri nesneyi bulur; IR'de tek bir nesne değişince kod diff'i de yalnızca o grubu etkiler.
- **İleride animasyon.** Animasyon komutları nesneler üzerinde çalışır (`Create(axes)`, `Write(label_vertex)`). İki karede aynı id varsa compiler `Transform` üretebilir; yeni id `FadeIn`, kaybolan id `FadeOut` olur. Böylece IR yeniden tasarlanmadan durağan karelerden animasyona geçilebilir.

Her backend bir yetenek listesi bildirsin

Backend, doğal olarak neyi desteklediğini belirtir (kübik Bézier, dash, LaTeX, raster görsel…). Front-end önce kütüphaneden bağımsız LIR'i üretir (`lir.json`); ayrı bir indirme adımı onu seçilen backend'in listesine göre basitleştirir: kübik eğri → kuadratik eğriler ya da kısa doğru parçaları, formül → glif path'leri. Bölge işlemleri, clip ve 3D projeksiyon bu listede yok, çünkü her zaman front-end'de hesaplanır (§2.6, §5). Backend'i orchestrator seçer; ajan hangi backend'in kullanıldığını bilmez. Bir özellik hiçbir şekilde indirilemiyorsa ajan bunu doğrulamada, backend adı geçmeden IR terimleriyle öğrenir.

Manim backend'i gerçekten kod üretmeli

ir3'teki `draw_manim.py` mobject'leri çalışma anında kuruyor; çıktı klasörüne konacak bir `scene.py` yok. Öneri:

- LIR'den, kendi başına çalışan ve okunabilir bir `scene.py` üretilsin. Manim CE sürümü sabitlensin; dosya `manim -s scene.py Frame` ile çalışsın.
- Her IR nesnesi için id'li bir fonksiyon veya `VGroup` olsun, üstünde `# ir: objects[7] id=v` gibi bir yorum dursun.
- Birebir karşılığı olan tiplerde, LIR'deki şekil ipucuyla Manim'in kendi sınıfları kullanılsın (`Circle` , `Line` , `MathTex`); konum ve boyut ise her zaman front-end'in hesapladığı açık sayılar olsun.
- Kütüphanenin kendi otomatik yerleşimini yapan parçalar (örneğin `Axes` 'in tikleri ve sayıları) primitive'lere indirilsin.
- Backend hatalarını nesnelere eşlemek için ayrı bir harness `scene.py` 'yi import edip her nesne fonksiyonunu ayrı ayrı çağırsın; böylece üretilen kod temiz kalır. Bu harness'i ajan değil, orchestrator çalıştırır.

SVG referans backend'i ilk günden olsun

LIR → SVG → PNG yazmak küçük bir iş ve üç fayda sağlar:

- ajan döngüsü için hızlı önizleme (Manim + LaTeX'ten çok daha hızlı);
- IR'nin gerçekten kütüphaneden bağımsız olduğunu ilk günden test eder;
- Manim çıktısını piksel farkıyla karşılaştırmak için bir altın referans olur; metin bölgelerinde küçük bir tolerans gerekir, çünkü her backend metni kendi font motoruyla çizer.

Backend uygunluk testleri

Çizgi kalınlığı, dash, delikli dolgu, baseline, z sırası ve opaklık için testler yazılsın. STATUS.md'deki "Lessons from building the second compiler" listesi (ir3'te Motion Canvas'tan sonra yazılan Manim compiler'ının dersleri) bu testlerin taslağı gibi. Kısa kenar her oranda 9 birim olduğu için referans px ile birim arasındaki oran sabit kalır (120 px/birim; Manim'de `stroke_width` = px / 1.2). Yine de katsayı koda gömülmesin, canvas'tan türetilsin; testler bunun her oranda ve her çıktı çözünürlüğünde (ör. 4K) doğru uygulandığını kontrol etsin.

Metin ölçümü

Backend'e bağımlılığın en büyük kaynağı bu: metnin genişliği fonta ve onu çizen motora bağlı (Manim'de metin için Pango, formül için LaTeX). Yerleşim front-end'de yapıldığı için front-end metni, hedef backend'in kullanacağı fontlar ve LaTeX şablonuyla ölçmeli; yoksa etiketler bir backend'de doğru, ötekinde kayık çıkar. Bu yüzden ölçüm değiştirilebilir bir arayüzün (`TextMetrics`) arkasında dursun; Manim için Pango ve Manim'in LaTeX şablonu kullanılır. LIR'den sonraki indirme adımı dışında, front-end'in hedef backend'e bağlı tek parçası budur. Hangi sağlayıcının kullanıldığı `meta.json` 'a yazılsın. Ajanın `measure_text` / `measure_tex` araçları da bu arayüzü çağırır; formülü ölçmek onu derlemeyi gerektirdiği için LaTeX hataları render'a gelmeden yakalanır.

4 IR ajanı ve kod çalıştırma

Ajan IR'yi script ile yazsın

Ajan sayıları ayrı bir script'te hesaplayıp sonra JSON'a elle kopyalarsa, kod çalıştırmanın önlemesi gereken hata (yanlış kopyalanan ya da yuvarlanan sayılar) geri gelir. Öneri, pydantic modelleri üzerinde ince bir `irkit` builder SDK:

- Ajanın script'i doğrudan `ir.json` üretir.
- Döngüler, sympy ile tam değerler (ör. `"44/17"`) ve geometri yardımcıları (kesişim, teğet, tohum noktası) bu script'te yazılır.
- Script varsa tek doğruluk kaynağı odur; düzeltmeler script'te yapılır ve script yeniden çalıştırılır.

Ajanın araçları

araç	ne verir
`run_python`	script'i sandbox'ta çalıştırır (`build_ir.py` → `ir.json`)
`validate`	şema ve anlam hataları; kod + JSON yolu + ipucu
`measure_text` / `measure_tex`	metin ve formül boyutları; hedef backend'in fontları ve LaTeX şablonuyla ölçülür
`resolve`	çözülmüş konum ve kutu tablosu, örneğin "etiket nereye düştü?"
`check`	lint sonuçları + SVG referans backend'inden id etiketli önizleme
`submit`	IR'yi teslim eder; doğrulamadan geçmeyen IR kabul edilmez

`run_python` dışındaki araçlar front-end'in fonksiyonları ve hiçbiri Manim'i çalıştırmaz. Ölçüm araçları hedef backend'in fontlarını kullanır ama ajan bunu bilmez; sonuç her zaman aynı biçimde, IR birimleriyle döner. Derleme ve Manim render'ı ajanın aracı değil, teslimden sonra orchestrator'ın çalıştırdığı bir pipeline adımı; bir sorun çıkarsa ajana §1'deki yönlendirmeyle, IR terimleriyle döner. Bu ayrım bilinçli: render ajanın aracı olsaydı ajan Manim'e bağlanır, IR'yi Manim'in kusurlarına göre ayarlamaya başlayabilir ve her turda yavaş render'ı beklerdi. SVG önizlemesiyle Manim çıktısı arasındaki fark bir backend bug'ıdır; onu backend uygunluk testleri yakalar (§3).

Sandbox şart

- LLM'in yazdığı Python ağ erişimi olmadan, yalnızca run klasörüne yazabilen ve CPU, bellek, süre sınırları olan bir izolasyonda çalışmalı.
- IR her zaman veri olarak kalmalı. İçindeki ifadeler güvenli bir mini dille değerlendirilir; `eval` kullanılmaz.
- LLM hiçbir zaman Manim kodu yazmaz; o kodu compiler üretir.

Prompt boyutu

IR.md şimdiden yaklaşık 40 KB. Spec şemadan otomatik üretilsin. Çekirdek kısım prompt'ta dursun; bileşen belgeleri gerektiğinde okunacak dosyalarda ve SDK docstring'lerinde kalsın.

5 3D hazırlığı ve çakışma semantiği

Şimdiden alınacak kararlar

Şimdi uygulamadan, ama ileride kapıyı kapatmayacak kararlar:

- `space` **genelleşsin:** `{"id", "kind": "cartesian2d" | ileride "3d", camera, projection}`. Noktalar uzayın boyutunda yazılır. Projeksiyonu front-end yapsın; LIR 2D + derinlik olarak kalsın. Böylece her 2D backend 3D bir projeksiyonu çizebilir ve Manim'in `ThreeDScene`'ine bağımlı kalmazsınız.
- **Derinliğe şimdiden yer açın:** LIR'de her primitive'e opsiyonel bir `depth` alanı (§3); IR'deki nesnelerin `z` alanı zaten öneriliyor (§2.8).

Çakışma: üç ayrı soru

soru	nerede kontrol edilir	2D'de	3D'de
Okunabilirlik	her zaman ekran uzayında	metin örtüşmesi, metnin üstünden geçen çizgi, kadraj dışı, kontrast	aynısı, projeksiyondan sonra
Şekil–şekil	2D'de ekran uzayı; 3D'de dünya uzayı	beklenmedik dolgu örtüşmesi uyarıdır	projeksiyondaki örtüşme normaldir (öndeki arkadakini örter). Gerçek sorunlar dünya uzayında aranır: istenmeden iç içe geçen katı cisimler, aynı düzlemde çakışan yüzeyler. Odak nesnesinin büyük kısmının örtülmesi ise ekranda kontrol edilir.
Kasıtlı temas	IR'deki ilişkiler	referansla bağlı nesneler muaf	aynısı

Kasıtlı temas üzerine: ir3'ün kontrolü okunabilirlik ve kompozisyonla ilgileniyor: kesişen çizgiler, birbirine değen şekiller ve uç uca vektörler uyarı üretmiyor; metne değen çizgiler ve dolgulu şekillerin örtüşmesi ise kontrol ediliyor. Bazı durumlar otomatik muaf: nesne ile kendi etiketi (eğri üstündeki etiket hariç), ucu bir metne bağlı çizgi (ör. açıklamadan çıkan ok), network'ün iç parçaları, şekil üstündeki noktalar, panel ile içindekiler. Geri kalanı `allow` ile elle bildiriliyor. Bunu genelleştirin: A nesnesi B'ye referansla tanımlandıysa (`"tail": "v.tip"`, `{"on": …}` ya da aynı grup), aralarındaki temas kasıtlı sayılsın; ir3'teki muafiyetlerin çoğu bu kuralın özel halleri. Muafiyet temas ve örtüşme uyarısını kaldırır; bir çizginin metnin üstünden geçip onu okunmaz hale getirmesi ise yine raporlanır. `allow` gerçek istisnalar için kalsın.

6 Korunması gerekenler

ir3'te doğru yapılmış olanlar olduğu gibi kalmalı:

- katı şema; her hatada kod, JSON yolu ve düzeltme ipucu;
- tek bir issue-code kaydı;
- `allow` + gerekçe;
- kontrollerin ayrı bir simülatörde değil, gerçekten çizilen geometri üzerinde çalışması;
- `intent` + `focus` alanları ve kavram renkleri;
- tam değerler için ifade dili;
- SEMANTICS.md disiplini;
- STATUS.md'deki dersler: girdiyi değiştirmemek, alan kurallarını tipe göre yazmak, paylaşılan önbelleklere kilit koymak.

Ayrıca 169 eski run'ın IR'leri, 0.1 → 1.0 dönüştürücüsüyle regresyon testi olarak kullanılabilir; 200 plan da `frame_brief` biçimine çevrildikten sonra eval korpusu olur.

7 Karar vermeniz gereken noktalar

#	karar	önerim
1	Backend'ler basit çizici mi olsun (tüm yerleşim front-end'de; geometri her kütüphanede aynı, yalnızca metin gliflerinde font motoruna bağlı küçük farklar olabilir), yoksa her kütüphanenin kendi otomatik yerleşimini kullanan, o kütüphaneye özgü kod mu üretilsin?	Basit çizici. Uygun yerlerde kütüphanenin kendi sınıfları kullanılsın, ama konumlar her zaman front-end'den gelsin.
2	Ajan IR'yi JSON olarak mı yazsın, SDK script'i ile mi?	SDK script'i; JSON sözleşme olarak kalır.
3	Planner'ın kararları (canvas, altyazı, font) IR'ye placeholder olarak mı girsin, kilitli bir header olarak mı?	Kilitli header; nesneler header'daki adlara referans verir (§1). Canvas'ın IR'de olması ir3'ün tersi, sizin tasarımınızla aynı.
4	Sıfırdan yeni bir paket mi, yoksa ir3'ün `py/irpic` kodunu mu geliştirelim?	Yeni ve katmanlı bir paket. ir3'ün algoritmaları taşınır: ifade değerlendirici, ölçüm kalibrasyonu, etiket yerleşimi ve lint kuralları front-end'e; network ve contour bileşen olarak.
5	Hangi altyazı modları olsun, metni kim yazsın?	`none` / `reserve` / `burn_in`; metni planner yazar.
6	Hangi fontlar seçilebilsin; planner tek bir punto mu versin, bir tip ölçeği mi?	Kurulu ve lisanslı "metin + matematik" çiftlerinden bir beyaz liste; tek punto yerine tip ölçeği (§2.5).
7	İçerik dili ne olacak?	`lang` header'da; içerik Türkçeyse LaTeX şablonu buna göre kurulur.
8	Planner ve IR ajanı hangi modelle çalışsın?	Pedagoji resmin kalitesini belirlediği için planner'ı daha güçlü bir modelle de deneyin. ir3'te IR ajanı claude-haiku-5-5 ile çalışıyordu; araçlarla birlikte Haiku'da kalabilir. Kararı eval versin.

8 Uygulama sırası

Tartışma sonrasında önerilen sıra:

1. **Şemalar:** `render_profile`, `frame_brief`, IR 1.0 ve LIR. Pydantic modellerinden JSON Schema ve spec belgesi üretilir. Bir de 0.1 → 1.0 dönüştürücüsü.
2. **Front-end iskeleti:** metin ölçümü, geometri çekirdeği (region, flood_fill, clip), SVG referans backend'i ve kontroller.
3. **Manim kod üretici** ve uygunluk testleri (SVG ile piksel farkı karşılaştırması).
4. **IR ajanı ve orchestrator döngüsü:** SDK, sandbox, header'ı kilitleme, teslimden sonra derleme, hata yönlendirme ve çıktı klasörü. Planner henüz yokken brief'ler elle yazılır.
5. **Planner** ve planner'a geri dönüş yolu.
6. **Eval matrisi:** alanlar × oranlar × altyazı modları × fontlar × backend'ler. Alanlar matematik, fizik, kimya, biyoloji, bilgisayar bilimi, tarih, coğrafya ve dil gibi geniş tutulmalı. Mevcut 200 plan STEM ağırlıklı ve 198'i dikey; tarih ya da zaman çizelgesi gibi konular hiç yok, ekonomi yalnızca iki kişisel finans planında var. Overfit ancak modelin görmediği alanlarda ölçülebilir.

Ek: Terimler

terim	anlamı
IR (ara temsil)	Resmin kütüphaneden bağımsız JSON tarifi; header'ı orchestrator, gövdeyi ajan yazar.
LIR (alt seviye IR, çizim listesi)	Front-end'in ürettiği, tüm konumları çözülmüş alt seviye temsil; backend'lerin okuduğu tek şey.
Front-end	IR'yi doğrulayan, metni ölçen, yerleşimi ve geometriyi hesaplayan, kontrolleri çalıştıran ortak katman.
Backend	LIR'i belirli bir kütüphaneyle çizen katman (Manim, SVG...).
Orchestrator	Pipeline'ı yöneten kod (LLM değil): header'ı brief'ten yazar, ajanı çalıştırır, teslimden sonra derlemeyi yapar ve hataları yönlendirir.
Lowering (indirme)	Bir özelliği, backend'in desteklediği daha basit primitive'lerle ifade etmek.
Anchor	Bir nesnenin hesaplanmış noktası (`v.tip` , `eq.bottom`); başka nesneler buna referans verir.
Lint	Çözülmüş geometri (LIR) üzerinde okunabilirlik ve kompozisyon kontrolleri (örtüşme, kadraj dışı, kontrast...).
Issue code	Her hata ve uyarı türünün tek bir kayıtta tanımlanmış kodu (ör. `E-SUBTITLE`).
`allow`	Ajanın bir uyarıyı gerekçesini yazarak bilinçli olarak kabul etmesi (ör. kasıtlı örtüşme).
READY	Resmin hazır sayılma koşulu: hata yok ve her uyarı düzeltilmiş ya da `allow` ile kabul edilmiş (ileride: görsel eleştirmen onayı).
Görsel eleştirmen	İleride eklenecek, son resme bakıp okunabilirliği ve pedagojiyi değerlendiren görsel dil modeli.
Bileşen (component)	Çekirdek nesnelere açılan, kendi şeması olan yardımcı tip (axes, graph, table...).
`defs`	Tanımlı ama kendisi çizilmeyen şekiller; operand, sembol ya da şablon olarak kullanılır.
Yetenek listesi	Bir backend'in doğal olarak desteklediği özelliklerin listesi (capability manifest).
Sandbox	Ajanın yazdığı kodun ağ ve dosya erişimi kısıtlanmış, izole bir ortamda çalışması.
Spec gap	Ajanın IR ile ifade edemediğini bildirdiği ihtiyaç; IR'nin neyi eksik bıraktığının kaydı.
Token	ir3'te renk, punto ve çizgi kalınlığı için izin verilen küçük ad listeleri; ajan yalnızca bunlardan seçebiliyordu.
Zone	ir3'te nesne yerleştirmek için adlandırılmış, sabit ekran bölgeleri.
SEMANTICS.md disiplini	Spec'in açık bıraktığı her davranışın SEMANTICS.md'ye yazılması ve tüm backend'lerin ona uyması.